User Guide: MATLAB Code for Vision-Based Motion Analysis

Overview

This MATLAB function processes a sequence of images to evaluate positioning repeatability by tracking the motion of a probe shadow. It calculates displacement in X and Y directions, radial deviation, and statistical error metrics, and exports results to text and CSV files.S # User Guide: MATLAB Code for Vision-Based Motion Analysis

Overview

This MATLAB function processes a sequence of images to evaluate positioning repeatability by tracking the motion of a probe shadow. It calculates displacement in X and Y directions, radial deviation, and statistical error metrics, and exports results to text and CSV files.

---

1. Preparing Your Data

Image Requirements

- Images must be:
  - In **.bmp format**
  - Sequentially named (e.g., `movexyp1.bmp`, `movexyp2.bmp`, ..., `movexyp30.bmp`)
- All images should be stored in a **single directory**

Update Image Path

Modify the file path in the code to match your image location:

```
file = sprintf('YOUR_FOLDER_PATH/movexyp%d.bmp', i);
```

Example:

```
file = sprintf('C:/Users/YourName/Data/movexyp%d.bmp', i);
```

---

2. Setting Key Parameters

At the beginning of the script, update the following variables based on your experiment:

Number of Images

```
n = 30;
```

- Set this equal to the total number of frames/images.

Pixel-to-Micron Conversion

```
pixel2Micron = 1.55;
```

- Replace with your calibration value.

Confidence Parameters

```
student_coeficient_value = 2.0452;
confidencelevel = 0.95;
```

- Modify only if using a different confidence level or sample size.

---

3. Output File Configuration

Text File Output

```
resultfile = fopen("YOUR_PATH/results.txt","w");
```

Example:

```
resultfile = fopen("C:/Users/YourName/results.txt","w");
```

CSV File Output

```
writetable(motionData, 'YOUR_PATH/results.csv');
```

Example:

```
writetable(motionData, 'C:/Users/YourName/results.csv');
```

⚠️ Ensure the folder exists before running the code.

---

4. Running the Code

1. Open MATLAB
2. Place the script in your working directory
3. Run the function:

```
sub_img_moveXY
```

---

5. What the Code Does (Step-by-Step)

Image Processing

- Reads all images into memory
- Converts images to grayscale
- Detects motion using pixel-wise differences
- Applies thresholding and noise removal

Object Detection

- Converts images to binary using adaptive thresholding
- Extracts the probe shadow
- Computes centroid coordinates

Displacement Calculation

- Calculates:
  - `dx` → horizontal displacement
  - `dy` → vertical displacement
  - `r` → radial displacement (in microns)

---

6. Statistical Analysis

The code computes:

- Mean radial deviation

- Standard deviation
- Absolute error
- Mean absolute error
- Standard mean deviation
- Random error (using Student's coefficient)
- Mean square error

---

7. Outputs Generated

Text File

Contains:

- Frame-by-frame displacement values
- Centroid coordinates
- Summary statistics

CSV File

Contains:

- Frame number
- dx, dy (in µm)
- Radial displacement (r)
- Absolute errors

Also appends:

- Mean value
- Standard deviation
- Random error

Plots

The code generates:

- Motion detection visualization
- Histogram comparison
- Polynomial fit of motion radius

- Polar scatter plot of displacement

---

8. Customization Options

Change Threshold Sensitivity

```
bw = diffImage > 0.000001;
```

- Increase value → less sensitive
- Decrease value → more sensitive

Noise Removal

```
bw = bwareaopen(bw, 50);
```

- Adjust `50` to remove smaller/larger noise regions

Polynomial Fit Degree

```
p = polyfit((1:n-1)', r, 5);
```

- Change `5` to adjust fitting complexity

---

9. Common Issues

No Centroid Detected

- Check image contrast
- Adjust thresholding
- Ensure probe is clearly visible

File Not Found Error

- Verify file paths
- Check naming format

Empty or NaN Values

- Indicates detection failure in some frames

---

10. Final Notes

- Ensure consistent imaging conditions for accurate results
- Verify calibration (pixel-to-micron conversion) before analysis
- Review plots to validate motion behavior

This code provides a complete pipeline for analyzing positioning repeatability and can be adapted for different trajectories by modifying input image datasets.

1. Preparing Your Data

Image Requirements

- Images must be:
  - In **.bmp format**
  - Sequentially named (e.g., `movexyp1.bmp`, `movexyp2.bmp`, ..., `movexyp30.bmp`)
- All images should be stored in a **single directory**

Update Image Path

Modify the file path in the code to match your image location:

```
file = sprintf('YOUR_FOLDER_PATH/movexyp%d.bmp', i);
```

Example:

```
file = sprintf('C:/Users/YourName/Data/movexyp%d.bmp', i);
```

2. Setting Key Parameters

At the beginning of the script, update the following variables based on your experiment:

Number of Images

```
n = 30;
```

- Set this equal to the total number of frames/images.

Pixel-to-Micron Conversion

```
pixel2Micron = 1.55;
```

- Replace with your calibration value.

Confidence Parameters

```
student_coeficient_value = 2.0452;
confidencelevel = 0.95;
```

- Modify only if using a different confidence level or sample size.

3. Output File Configuration

Text File Output

```
resultfile = fopen("YOUR_PATH/results.txt","w");
```

Example:

```
resultfile = fopen("C:/Users/YourName/results.txt","w");
```

CSV File Output

```
writetable(motionData, 'YOUR_PATH/results.csv');
```

Example:

```
writetable(motionData, 'C:/Users/YourName/results.csv');
```

⚠️ Ensure the folder exists before running the code.

4. Running the Code

1. Open MATLAB
2. Place the script in your working directory
3. Run the function:

```
sub_img_moveXY
```

5. What the Code Does (Step-by-Step)

Image Processing

- Reads all images into memory
- Converts images to grayscale

- Detects motion using pixel-wise differences
- Applies thresholding and noise removal

Object Detection

- Converts images to binary using adaptive thresholding
- Extracts the probe shadow
- Computes centroid coordinates

Displacement Calculation

- Calculates:
  - `dx` → horizontal displacement
  - `dy` → vertical displacement
  - `r` → radial displacement (in microns)

6. Statstical Analysis

The code computes:

- Mean radial deviation
- Standard deviation
- Absolute error
- Mean absolute error
- Standard mean deviation
- Random error (using Student's coefficient)
- Mean square error

7. Outputs Generated

Text File

Contains:

- Frame-by-frame displacement values
- Centroid coordinates
- Summary statistics

CSV File

Contains:

- Frame number

- dx, dy (in µm)
- Radial displacement (r)
- Absolute errors

Also appends:

- Mean value
- Standard deviation
- Random error

Plots

The code generates:

- Motion detection visualization
- Histogram comparison
- Polynomial fit of motion radius
- Polar scatter plot of displacement

8. Customization Options

Change Threshold Sensitivity

```
bw = diffImage > 0.000001;
```

- Increase value → less sensitive
- Decrease value → more sensitive

Noise Removal

```
bw = bwareaopen(bw, 50);
```

- Adjust `50` to remove smaller/larger noise regions

Polynomial Fit Degree

```
p = polyfit((1:n-1)', r, 5);
```

- Change `5` to adjust fitting complexity

9. Common Issues

No Centroid Detected

- Check image contrast
- Adjust thresholding
- Ensure probe is clearly visible

File Not Found Error

- Verify file paths
- Check naming format

Empty or NaN Values

- Indicates detection failure in some frames

10. Final Notes

- Ensure consistent imaging conditions for accurate results
- Verify calibration (pixel-to-micron conversion) before analysis
- Review plots to validate motion behavior

This code provides a complete pipeline for analyzing positioning repeatability and can be adapted for different trajectories by modifying input image datasets.